RAPPORT D'AUDIT DE PERFORMANCE

BarMash

Moteur de simulation temps réel en Go — 10 000 agents, 30 ticks/s

PÉRIMÈTRE DE L'AUDIT

BarMash simule une soirée dans un bar : 10 000 clients se déplacent vers 8 comptoirs, boivent, se bousculent et laissent des flaques qui ralentissent leurs voisins. Le serveur calcule un tick toutes les 33,3 ms et pousse l'état complet au navigateur par WebSocket.

L'audit part de la version naïve livrée, mesure ses goulots réels, puis applique et vérifie des optimisations sur trois axes : calcul et mémoire, concurrence, I/O réseau et persistance. Jusqu'à la parallélisation, chaque étape reproduit l'état de la simulation au bit près. La parallélisation impose ensuite un changement de modèle, documenté et vérifié.

Réalisé par **Paul-Antoine CARREAU** • Master 2 Expert en Développement Logiciel (M2 Dev B) **18 octobre 2026**

1. Banc d'essai et protocole de mesure

Toutes les mesures ont été faites sur la même machine fixe, **branchée sur secteur, redémarrée juste avant la campagne, sans aucun autre programme démarré à côté**. La baseline et la version finale sont mesurées dans la même session, l'une après l'autre.

Matériel et logiciel

COMPOSANT	SPÉCIFICATION	IMPACT SUR L'AUDIT
CPU	AMD Ryzen 7 9700X (Zen 5) — 8 cœurs / 16 threads, 3,8 GHz (boost 5,5 GHz)	Baseline mono-thread ; version finale : 8 workers, un par cœur physique
Caches	L1d 48 Ko + L1i 32 Ko par cœur • L2 1 Mo par cœur • L3 32 Mo partagé	Les données chaudes (positions : 160 Ko) tiennent en L2
Mémoire	32 Go DDR5-5600 (2 × 16 Go Micron, CL36)	Pas de contrainte de capacité
Système	Windows 11 Professionnel (build 26300), x86_64	Mesures sous Windows, pas sous Linux
Runtime	Go 1.27.0 windows/amd64 (compilateur gc, inlining et escape analysis activés)	Versions identiques pour baseline et final
Outils	benchstat (x/perf) • Hyperfine 1.20.0 • go tool pprof • fieldalignment	

Protocole

MESURE	MÉTHODE	PARAMÈTRES
Coût d'un tick	`go test -bench`, 1 op = 1 tick, état restauré depuis un snapshot tous les 100 ticks pour une charge stable	n = 10 séries • benchstat (Mann-Whitney), seuil p < 0,05
Coût d'une frame réseau	Télémétrie + encodage de la frame complète (10 000 clients, ~360 Ko de JSON)	idem
Bout en bout	Hyperfine sur le binaire en mode headless : 5 000 ticks de simulation, du lancement à la sortie	`--warmup 3 --runs 10 -N` (sans shell intermédiaire)
Profils	pprof CPU et allocations sur les mêmes 5 000 ticks, plus un profil live de 20 s sur le serveur réel	Échantillonnage 100 Hz
I/O	Frame binaire vs JSON ; SQLite sur disque (WAL) avec 200 000 lignes d'historique ; handler HTTP	n = 10 • benchstat
Non-régression	Empreinte FNV-1a de tout l'état (positions, alcoolémie, timers, flaques, RNG) après 4 000 ticks	Jusqu'à O8 : `0xaa36…614c` (naïf) • depuis O9 : `0xfcd1…c638`, quel que soit le nombre de workers

Deux scénarios de charge

Le premier constat de l'audit concerne le benchmark fourni : il ne mesurait que le début de soirée, quand aucune flaque n'existe. Or les flaques n'apparaissent qu'après ~2 500 ticks et leur coût domine ensuite. Deux scénarios figés ont donc été créés.

SCÉNARIO	ÉTAT	TICK NAÏF
early — début de soirée	Tick 0, aucune flaque	1,539 ms
rush — régime de croisière	Tick 5 000, ~450 flaques actives, 214 clients en train de vomir	5,934 ms

Baseline mesurée avec Hyperfine (5 000 ticks, version naïve)

15,29 s	14,83 s	± 1,00 s	1,00 s²
Moyenne	Médiane	Écart-type (σ)	Variance (σ²)

Min 14,35 s • max 17,11 s • coefficient de variation 6,5 %. La version finale reste à 0,7 % dans la même session (2,24 ± 0,02 s). La dispersion de la baseline vient de quelques runs plus lents (max 17,1 s) : le binaire naïf déclenche 647 cycles de GC concurrents sur les autres cœurs, ce qui le rend sensible aux variations de fréquence. La médiane (14,83 s) sert donc de référence.

2. Diagnostic de la baseline

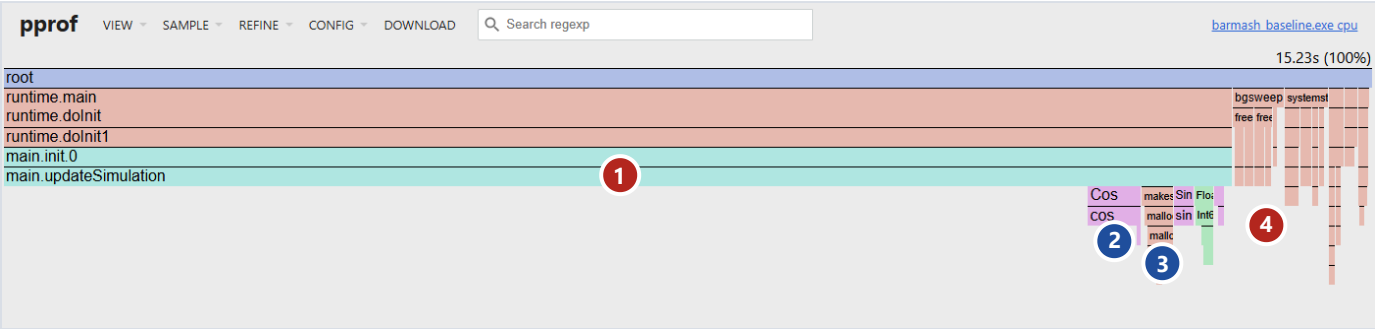


Figure 1 — Flamegraph CPU réel de la version naïve (pprof, 5 000 ticks headless, `perf/campaign/baseline_cpu.pprof`). Total 15,23 s. Repères : (1) plateau de `updateSimulation`, où les boucles des flaques et de la bousculade sont écrites en ligne (70,6 % à elles deux) ; (2) `Cos` / `Sin` ; (3) `makeslice` / `mallocgc` ; (4) tours du GC (`bgsweep`, `gcBgMarkWorker`).

Le flamegraph ne montre qu'un grand plateau, car les boucles chaudes sont écrites en ligne dans `updateSimulation`. Le détail vient donc du profil ligne à ligne :

REPÈRE	ZONE DU HOT PATH	5 000 TICKS	RUSH	CAUSE IDENTIFIÉE
1	**Scan des flaques** — chaque client teste chaque flaque	**38,6 %**	**~60 %**	O(N × P) : 10 000 × ~450 = 4,5 M tests et racines carrées par tick
1	**Scan de bousculade** — 60 voisins par client	**32,0 %**	**~33 %**	600 k lectures de `*Client` de 96 o via pointeurs ; `math.Sqrt` sur tous les voisins
2	Trigonométrie du déplacement	5,4 %	~5 %	`math.Cos` + `math.Sin` séparés pour chaque client mobile
3	Allocations dans la boucle	2,3 %	~2 %	`makeslice` : `make([]BarDistance)`, 99,2 % des octets alloués (profil ci-dessous)
4	Garbage Collector (balayage + marquage)	4,7 %	—	`bgsweep` 3,8 % + `gcBgMarkWorker` 0,9 % : conséquence directe de 3
	Recherche du bar le plus proche	2,9 %	—	8 racines carrées par client assoiffé

Part du temps CPU total (15,23 s) attribuée ligne à ligne avec `pprof -top -lines`. Le scan des flaques pèse moins sur 5 000 ticks parce que la première moitié de la soirée n'en contient aucune.

Mémoire et Garbage Collector

INDICATEUR (VERSION NAÏVE)	VALEUR	ORIGINE
Allocations par tick	426 Ko • 3 405 allocs	`make([]BarDistance, len(bars))` par client assoiffé (taille non constante → tas)
Allocations par frame réseau	574 Ko • 6 allocs	`[]ClientPoint` + `json.Marshal` + conversion `string(data)`
Coût d'une frame réseau	1,185 ms	Réflexion de `encoding/json` sur 10 000 structures
5 000 ticks sans serveur	647 cycles GC • 1 820 Mo alloués	Soit ~364 Ko de déchets par tick, collectés en continu (repère 4)
Serveur réel, 25 s	493 cycles GC	S'y ajoute `runtime.ReadMemStats` (pause Stop-The-World) à chaque tick

```
# Profil d'allocations réel de la baseline (5 000 ticks), attribué par ligne de source
$ go tool pprof -sample_index=alloc_space -top -lines barmash_baseline.exe baseline_allocs.pprof
      flat  flat%    cum   cum%
    1.78GB 99.20%  1.78GB 99.20%  main.updateSimulation main.go:409   ← barDistances := make([]BarDistance, len(bars))
# alloc_objects : 14 915 356 objets (99,68 %) sur la même ligne — une tranche écrite mais jamais relue
```

Lecture

- Le goulot n°1 est **algorithmique** (scan des flaques), pas mémoire : il ne pèse rien en début de soirée et passe à 60 % du temps en croisière.
- Le goulot n°2 combine calcul (racines carrées inutiles) et accès mémoire indirects.
- Côté réseau, la frame coûte presque autant qu'un tick et génère l'essentiel des déchets mémoire.

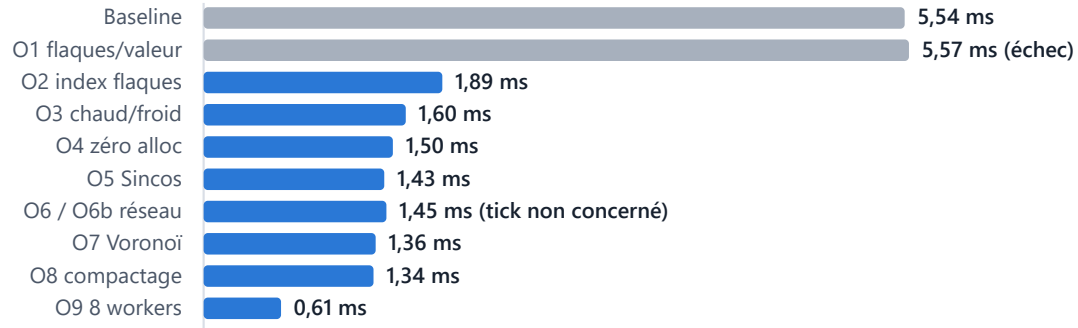
3. Journal d'optimisation

Les actions sont ordonnées selon la loi d'Amdahl, en commençant par la zone qui pèse le plus. Chaque ligne correspond à un commit et à une série benchstat (n = 10) comparée à l'étape précédente. « ~ » signale un écart non significatif (p ≥ 0,05).

#	ACTION	HYPOTHÈSE MATÉRIELLE	EARLY	RUSH	VERDICT
Calcul CPU et algorithmique					
O2	Index spatial des flaques (grille 32 × 32, tri par comptage)	4,5 M tests → ~4 candidats par client	~	−66,1 %	retenu
O5	`math.Sincos` au lieu de `Cos` + `Sin`	Une seule réduction d'argument	−8,2 %	−4,6 %	retenu
O7	Table précalculée du bar le plus proche (zones de Voronoï)	8 racines carrées → 1 ; table de 6,4 Ko en L1	−8,9 %	−6,2 %	retenu
Mémoire et localité de cache					
O1	Flaques stockées par valeur, expiration FIFO	Supprimer les sauts de pointeurs	+1,4 %	+0,6 %	échec
O3	Positions dans un tableau contigu de 16 o + rejet par boîte englobante	160 Ko en L2 ; 97 % des voisins rejetés sans `Sqrt`	−21,0 %	−15,5 %	retenu
O4	Suppression des allocations par tick	Plus de pression GC dans la boucle	−9,9 %	−6,1 %	0 alloc
O8	Compactage de `Client` : 80 o → 48 o (tri des champs, types réduits)	Clients + positions (640 Ko) dans le L2	~	~	neutre
Concurrence et scalabilité CPU					
O9	Tick en deux phases : forces calculées en parallèle par 8 workers, machine à états séquentielle	Phase parallèle ≈ 60 % du tick ; distribution par compteur atomique, sans verrou ni faux partage	−54,2 %	−55,1 %	retenu
I/O réseau et persistance					
O6	Encodeur JSON écrit à la main, buffer réutilisé, `runtime/metrics`	0 déchet par frame, plus de pause Stop-The-World		frame −66,5 %	retenu
O6b	Table des entiers 0..2047 précalculée	40 000 entiers écrits en un store de 4 o		frame −60,5 %	retenu
O10	Frame binaire à champs fixes (6 o par client)	~6 × moins d'octets à écrire et à envoyer		−78 % • 360 → 61 Ko	retenu
O11	Historique SQLite : insertions groupées en transaction	1 commit pour 100 lignes au lieu de 100		36,1 → 3,6 µs/ligne	×10
	Index composite `(session_id, tick)`	`SCAN` de 200 000 lignes → `SEARCH USING INDEX`		11,47 → 0,28 ms	×40
	`sync.Pool` sur le buffer de réponse `/api/history`	Buffer déjà dimensionné, réutilisé d'une requête à l'autre		559 → 57 Ko/requête	−90 %

Contrôles : empreinte bit à bit identique au naïf jusqu'à O8, puis invariante au nombre de workers ; frame JSON octet-identique à `encoding/json` ; frame binaire décodée et comparée à l'état ; index, table de Voronoï et `Sincos` comparés à l'implémentation naïve sur des millions de points.

Évolution du coût d'un tick en croisière (scénario rush)



Médianes benchstat mesurées étape par étape. Deux actions font l'essentiel : O2 (−66 %, le plateau le plus large du profil) puis O9 (−55 %). Le comparatif final (section 6) est refait dans une même session.

4. Concurrence, tentatives infructueuses et reproductibilité

O9 — Pourquoi un changement de modèle

Dans la boucle naïve, le client *i* lit les positions déjà déplacées des clients précédents : chaque client dépend du précédent, ce qui empêche toute parallélisation exacte. La bousculade est donc calculée sur les positions de **début de tick** (phase 1, parallèle), puis la machine à états et le RNG s'exécutent dans l'ordre (phase 2).

- Résultat identique avec 1, 3, 8 ou 16 workers (test dédié).
- Écart d'observables après 5 000 ticks par rapport au naïf : bières −0,02 %, flaques −1,4 %, vomisseurs 217 contre 214.
- Arrêt précoce : contexte annulé dès la déconnexion du navigateur ; Ctrl+C arrête le serveur et vide le dernier lot SQLite.

Dimensionnement du worker pool

WORKERS	EARLY	RUSH
O8 (séquentiel)	0,915 ms	1,361 ms
1	+4,5 %	+7,8 %
2	−34,0 %	−10,1 %
4	−51,1 %	−42,9 %
8 (cœurs physiques)	**−54,2 %**	**−55,1 %**
16 (threads SMT)	−56,8 %	−57,6 %

16 workers ne gagnent que 5,6 % de plus que 8, pour deux fois plus de threads : les deux threads SMT d'un cœur partagent ses unités flottantes. On garde 8 workers et on laisse les autres threads au réseau, au GC et à SQLite.

Échec n°1 — Rejet par boîte englobante écrit avec `||` : +52 % de temps

VARIANTE (SCAN DE BOUSCULADE)	EARLY	RUSH
Avant (séparation chaud/froid seule)	1,40 ms	—
`if ndx >= 22 || ndx <= -22 { continue }`	2,28 ms (+52 %)	2,72 ms (+44 %)
`if math.Abs(ndx) >= 22 { continue }`	1,14 ms (−19 %)	1,60 ms

- **Cause :** le premier saut dépend du **signe** de dx, qui est aléatoire, et le prédicteur de branchement se trompe une fois sur deux. 600 k voisins × 50 % × ~15 cycles de vidage du pipeline ≈ **0,9 ms**, soit l'écart mesuré (+0,88 ms).
- **Correction :** `math.Abs` est un masque de bit sans branchement, qui ne laisse qu'un saut, juste dans 97 % des cas. La même condition coûte +60 % ou −19 % selon son écriture.

Échec n°2 — Paralléliser sans changer le modèle

- Seule la détection des flaques est parallélisable sans changer le résultat : ~10 % du tick.
- Mesure (8 workers) : −9,8 % rush, mais +5,0 % early, où il n'y a aucune flaque et où l'on paie la synchronisation pour rien.
- Amdahl : avec P ≈ 0,1, le gain est plafonné à 11 % même avec une infinité de cœurs. Abandonné au profit du modèle en deux phases.

Échec n°3 — Optimisations mémoire sans effet CPU

- **O1** (flaques par valeur) : +1,4 % / +0,6 %. Les flaques tenaient déjà en 18 Ko dans le L1d ; le vrai problème était le nombre de tests.
- **O8** (struct de 80 → 48 o) : p = 0,063 / 0,481. Les clients sont lus séquentiellement et le prefetcher suit le flux. Conservé pour l'empreinte mémoire (−40 %).

Reproduire la campagne en une commande

```
# Prérequis : Go 1.27, benchstat, hyperfine (vegeta en option). Durée ~16 min. Résultats dans perf/campaign/
./run_benchmarks.sh
```

Le script reconstruit la baseline depuis git (`343a4ee`) avec le même harnais headless, lance les tests, les benchmarks des deux versions dans la même session (benchstat), la montée en charge des workers, les mesures I/O et les plans de requête, Hyperfine, les profils pprof, puis le test de charge HTTP (`perf/load_test.sh`). Un commit par lot d'optimisation, séries brutes dans `perf/bench/` . Règles imposées aux assistants IA : `constitution.md` .

5. Validation sous charge et I/O

Test de charge HTTP de `/api/history` (Vegeta, 15 s par palier)

Requête : export complet d'une session (4 000 lignes, ~1 h de simulation) parmi 50, dans une base de 200 000 lignes. Même serveur, même requête ; seule change la présence de l'index `(session_id, tick)`.

BASE	DÉBIT DEMANDÉ	DÉBIT SERVI	SUCCÈS	P50	P95	P99	MAX
Avec index	**500 req/s**	**499,9 req/s**	**100 %**	**4,25 ms**	**7,22 ms**	**13,38 ms**	**39,3 ms**
Sans index	50 req/s	50,0 req/s	100 %	16,21 ms	18,78 ms	21,50 ms	38,7 ms
Sans index	500 req/s	9,2 req/s	5,5 %	30 s	30 s	30 s	30 s

Sans index, chaque requête parcourt les 200 000 lignes puis trie le résultat. Le serveur tient 50 req/s, mais s'effondre à 500 req/s : 94,5 % des requêtes tombent en timeout (30 s) et le débit servi chute à 9 req/s. Avec l'index, le même débit passe avec un p99 de 13 ms. Les latences sous la milliseconde ne sont pas mesurables sous Windows (résolution de l'horloge Go ~0,5 ms), d'où le choix d'une requête lourde.

```
# EXPLAIN QUERY PLAN — SQLite n'a pas d'EXPLAIN ANALYZE ; les durées viennent des benchmarks ci-dessus
sans index : SCAN telemetry | USE TEMP B-TREE FOR ORDER BY
avec index : SEARCH telemetry USING INDEX idx_telemetry_session_tick (session_id=? AND tick>? AND tick<?)
```

Frame binaire : pourquoi pas Protobuf

- Toutes les valeurs ont une taille fixe et connue (positions sur 16 bits, état sur 8 bits) : un format à offsets fixes se lit directement avec un `DataView`, sans parseur.
- Protobuf avec un message par client coûte ~12 o par client (étiquette + varint par champ) ; en champs `packed`, on retombe à ~6–7 o, soit la même taille, mais avec une bibliothèque, une étape de génération et un décodage varint côté navigateur.
- Le format est versionné (octet 0) ; les données statiques partent une fois dans un message `init`.

FRAME (CROISIÈRE)	TAILLE	ENCODAGE
JSON naïf (`json.Marshal`)	360 Ko	1,185 ms
JSON append-only (O6)	360 Ko	130 µs
Binaire (O10)	**61 Ko**	**29 µs**

Serveur réel : 25 s avec un client WebSocket

MESURE	NAÏF	FINAL
Frames reçues	30 /s	30 /s
Taille moyenne d'une frame	310 Ko	59 Ko
CPU consommé (profil live 20 s)	2 670 ms	990 ms
Cycles de GC	493	2

Avant la parallélisation, la version finale consommait 840 ms : les 8 workers ajoutent un peu de CPU cumulé (réveil et synchronisation à chaque tick) en échange d'une latence de tick divisée par 2,2. L'écriture SQLite, asynchrone, n'apparaît pas dans le profil du tick.

6. Synthèse et conclusion

×10,4	×41	−83 %	0 alloc
Tick en croisière plus rapide	Frame réseau plus rapide	d'octets envoyés au navigateur	par tick et par frame

Baseline vs version finale

INDICATEUR	BASELINE	FINALE	ÉCART	SOURCE
Tick, début de soirée (early)	1,539 ms	0,389 ms	−74,7 % (×4,0)	benchstat n=10
Tick, croisière (rush)	5,934 ms	0,569 ms	−90,4 % (×10,4)	benchstat n=10
Frame réseau (télémétrie + encodage)	1,185 ms (JSON)	29,0 µs (binaire)	−97,6 % (×41)	benchstat n=10
Taille d'une frame • débit à 30 frames/s	360 Ko • 10,8 Mo/s	61 Ko • 1,8 Mo/s	−83 %	benchstat (bytes/frame)
Allocations par tick • par frame	426 Ko • 574 Ko	0 • 0	−100 %	benchstat
5 000 ticks de bout en bout (moyenne ± σ)	15,29 ± 1,00 s	2,24 ± 0,02 s	×6,8	Hyperfine, 3 + 10 runs
GC sur 5 000 ticks • mémoire allouée	647 cycles • 1 820 Mo	0 cycle • 1,12 Mo	−100 %	runtime/metrics
Serveur réel, 25 s : CPU • cycles GC	2 670 ms • 493	990 ms • 2	−63 %	pprof live + WebSocket
Taille d'un client en mémoire	96 o (via pointeur)	48 o + 16 o	−33 %	`unsafe.Sizeof`
État après 5 000 ticks (bières • flaques)	**141 838 • 438**	**141 809 • 432**	**−0,02 % • −1,4 %**	**modèle en deux phases**

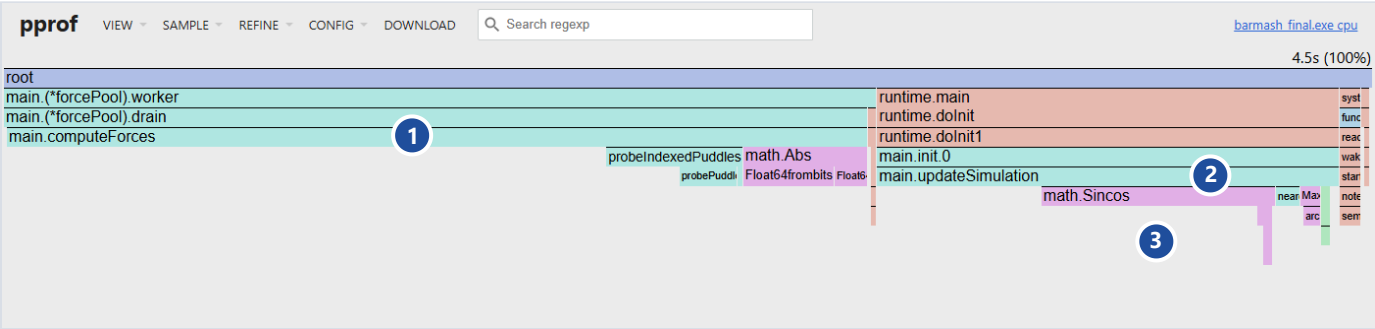


Figure 2 — Flamegraph CPU de la version finale sur les mêmes 5 000 ticks : 4,50 s de CPU cumulé pour 2,18 s de temps réel (contre 15,23 s). Repères : (1) phase parallèle dans les workers, `computeForces` (63 %) ; (2) phase séquentielle (34 %) ; (3) `math.Sincos` (17 %), premier poste de la partie séquentielle, qui borne désormais le gain (Amdahl). Plus aucune tour du GC.

Conclusion

Actions les plus rentables : l'index des flaques (O2), qui divise à lui seul le tick de croisière par 2,95, puis le tick en deux phases sur 8 workers (O9), qui le divise encore par 2,2. Côté réseau, la frame binaire coûte 41 fois moins cher à produire que le JSON d'origine, pour six fois moins d'octets.

Accent mis : d'abord le **CPU** (algorithme, branchements, racines carrées, parallélisme), puis la **mémoire et le GC** (zéro allocation, données sans pointeurs), enfin les **I/O** (octets envoyés, base indexée, écritures groupées et asynchrones). Le tick tient en 0,569 ms sur un budget de 33,3 ms.

Ce qui n'a pas marché : le rejet par boîte englobante écrit avec `||`. Une optimisation évidente sur le papier a ralenti le tick de 52 % à cause du prédicteur de branchement. Paralléliser sans toucher au modèle n'aurait rapporté que 10 % au mieux (loi d'Amdahl), et les optimisations purement mémoire (O1, O8) n'ont rien apporté au CPU : les données tenaient déjà en cache.

Limites : la parallélisation change légèrement la dynamique de la simulation (écart chiffré ci-dessus), un choix assumé et testé. Le détecteur de data races (`go test -race`) demande cgo et n'a pas pu être lancé sous Windows ; il est à passer en CI Linux.